

SIMULAÇÃO: COMPILADORES E BANKO DE DADOS COMPARTILHADOS

Questão 2 - Trabalho 01 de Sistemas Operacionais

IFCE - Campus Maracanaú | Prof. Daniel Ferreira

Data: 12 de May de 2026

ÍNDICE

- 1. Descrição do Problema
- 2. Importações e Bibliotecas
- 3. Variáveis de Sincronização
- 4. Variáveis de Estado
- 5. Funções Auxiliares
- 6. Função Principal (programmer_work)
- 7. Função Main
- 8. Características de Segurança
- 9. Como Executar

1. DESCRIÇÃO DO PROBLEMA

Cenário: Cinco programadores trabalham em um laboratório, cada um desenvolvendo módulos diferentes. Todos precisam compilar seus códigos, mas existem recursos limitados: **um compilador exclusivo** e **um banco de dados de dependências compartilhado** (máximo 2 acessos simultâneos).

Restrições:

- Apenas 1 programador pode usar o compilador por vez
- No máximo 2 programadores podem acessar o banco de dados simultaneamente
- Um programador só pode compilar quando tem ambos os recursos
- Deve-se evitar deadlocks (travamentos) e inanição (espera infinita)

Solução: Usar primitivas de sincronização do Python (threading) para garantir acesso ordenado aos recursos e evitar condições de corrida.

2. IMPORTAÇÕES E BIBLIOTECAS

O código utiliza as seguintes bibliotecas Python:

Biblioteca	Uso
threading	Criar múltiplas threads (programadores) que executam simultaneamente
time	Controlar delays e sleep() para simular tempos de compilação
random	Gerar tempos aleatórios de pensamento e compilação
datetime	Gerar timestamps formatados para visualizar sequência de eventos

Código:

```
import threading
import time
import random
from datetime import datetime
```

3. VARIÁVEIS DE SINCRONIZAÇÃO

Essas variáveis controlam o acesso aos recursos compartilhados, garantindo que múltiplas threads não acessem simultaneamente de forma descontrolada.

3.1 - `compiler_lock = threading.Semaphore(1)`

Um **Semáforo** é uma primitiva de sincronização que controla acesso a um recurso. O valor **1** significa que apenas 1 programador pode usar o compilador por vez.

- **Acquire()**: Programador tenta usar o compilador (bloqueia se ocupado)
- **Release()**: Programador libera o compilador para outro usar

3.2 - `database_semaphore = threading.Semaphore(2)`

Permite que **até 2 programadores** acessem o banco de dados simultaneamente, evitando sobrecarga. O terceiro fica aguardando.

3.3 - `state_lock = threading.Lock()`

Um **Lock (Mutex)** garante que apenas 1 thread por vez possa imprimir na tela, evitando que mensagens se sobreponham (saída confusa).

Código:

```
compiler_lock = threading.Semaphore(1)
database_semaphore = threading.Semaphore(2)
state_lock = threading.Lock()
```

4. VARIÁVEIS DE ESTADO

programmer_states: Dicionário que armazena o estado atual de cada programador.

Código:

```
programmer_states = {  
    f"P{i+1}": "Pensando" for i in range(5)  
}
```

Exemplo de conteúdo:

```
{  
    'P1': 'Pensando',  
    'P2': 'Compilando',  
    'P3': 'Aguardando compilador',  
    'P4': 'Acessou BD',  
    'P5': 'Pensando'  
}
```

Cada programador pode estar em um dos seguintes estados:

- **Pensando:** Descansando entre compilações
- **Aguardando acesso ao BD:** Esperando para acessar o banco de dados
- **Acessou BD, aguardando compilador:** Tem BD, mas espera pelo compilador
- **Compilando:** Usando o compilador e o BD simultaneamente
- **Compilação concluída:** Terminou a compilação
- **Liberou compilador e BD:** Liberou ambos os recursos

```

P1: Compilando (Ciclo 1)
P2: Aguardando compilador (Ciclo 1)
P3: Pensando (Ciclo 2)

```

P4: Acessou BD, aguardando compilador (Ciclo 1)

P5: Pensando (Ciclo 1)



6. FUNÇÃO PRINCIPAL - programmer_work()

Esta é a **função mais importante**. Cada programador executa esta função em uma thread separada, criando assim 5 execuções paralelas.

6.1 - Inicialização

Código:

```
def programmer_work(programmer_id):  
    cycle = 1  
    while True:
```

- **cycle:** Contador que rastreia quantas vezes o programador já compilou
- **while True:** Loop infinito para simular trabalho contínuo

6.2 - Fase 1: Pensando (Thinking)

Código:

```
print_state(programmer_id, f"Pensando (Ciclo {cycle})")  
time.sleep(random.uniform(1, 3))
```

- Imprime que o programador está pensando
- Aguarda entre 1 e 3 segundos (tempo aleatório)
- Simula o tempo que o programador descansa entre compilações

6.3 - Fase 2: Adquirir Banco de Dados

Código:

```
print_state(programmer_id, f"Aguardando acesso ao BD (Ciclo {cycle})")  
database_semaphore.acquire()
```

- Imprime que está aguardando o banco de dados
- **Semáforo bloqueia aqui** se 2 programadores já estão acessando o BD
- Quando um dos 2 libera, este programador prossegue

6.4 - Fase 3: Aguardar Compilador

Código:

```
print_state(programmer_id, f"Acessou BD, aguardando compilador (Ciclo {cycle})")  
compiler_lock.acquire()
```

- Imprime que tem acesso ao BD, mas aguarda compilador
- **Lock bloqueia aqui** se outro programador está compilando
- Quando o outro libera, este começa a compilar

6.5 - Fase 4: Compilar (Seção Crítica)

Código:

```
print_state(programmer_id, f"Compilando (Ciclo {cycle})")
compilation_time = random.uniform(2, 4)
time.sleep(compilation_time)
```

- **Seção Crítica:** Apenas este programador está compilando
- Tem acesso exclusivo ao compilador E ao BD
- Tempo de compilação varia entre 2 e 4 segundos
- Nenhum outro programador pode compilar durante este período

6.6 - Fase 5: Liberar Recursos

Código:

```
print_state(programmer_id, f"Compilação concluída (Ciclo {cycle})")
compiler_lock.release()
database_semaphore.release()
print_state(programmer_id, f"Liberou compilador e BD")
cycle += 1
```

- Libera o compilador para outro programador
- Libera um slot do banco de dados
- Imprime que liberou os recursos
- Incrementa o contador de ciclos
- Loop continua: volta ao passo 6.2 (pensando)

7. FUNÇÃO MAIN

Código:

```
def main():
    print("\n" + "="*60)
    print("SIMULAÇÃO: COMPILADORES E BANCO DE DADOS...")
    print("="*60)

    threads = []
    for i in range(5):
        programmer_id = f"P{i+1}"
        t = threading.Thread(
            target=programmer_work,
            args=(programmer_id, ),
            daemon=True
        )
        threads.append(t)
        t.start()
```

7.1 - Criar Threads

- **for i in range(5):** Cria 5 threads (programadores)
- **threading.Thread():** Cria uma nova thread
- **target=programmer_work:** Função que a thread executa
- **args=(programmer_id,):** Passa P1, P2, P3, P4 ou P5 como argumento
- **daemon=True:** Thread finaliza quando o programa principal fecha
- **t.start():** Inicia a execução da thread

7.2 - Loop Infinito Principal

Código:

```
try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    print("\n\nSimulação interrompida pelo usuário.")
```

- **while True:** Mantém o programa rodando indefinidamente
- **time.sleep(1):** Evita consumo excessivo de CPU
- **except KeyboardInterrupt:** Captura Ctrl+C para parar a simulação
- As 5 threads continuam executando em paralelo

7.3 - Ponto de Entrada

Código:

```
if __name__ == "__main__":
    main()
```

Garante que main() só é executada quando o arquivo é rodado diretamente, não quando importado por outro módulo.

8. CARACTERÍSTICAS DE SEGURANÇA

8.1 - Evitar Deadlock (Travamento)

Deadlock ocorre quando threads ficam esperando uma pela outra infinitamente. Exemplo: P1 segura BD e aguarda compilador, P2 segura compilador e aguarda BD.

Como evitamos:

- **Ordem fixa de aquisição:** Sempre BD primeiro, depois compilador
- **Mesmo para todas as threads:** Impossível fazer ciclo de espera
- **Semáforos e locks:** Garantem atomicidade de operações

8.2 - Evitar Inanição (Starvation)

Inanição ocorre quando uma thread nunca consegue acesso ao recurso (sempre fica esperando).

Como evitamos:

- **Semáforos do Python:** Implementam fila FIFO (First In First Out)
- **Justa atribuição:** Quem chegou primeiro tem prioridade
- **Sem prioridades:** Todas as threads têm mesma chance

8.3 - Race Condition na Saída

Sem sincronização, 5 threads tentando imprimir simultaneamente resultaria em saída entrelaçada e confusa.

Solução:

- **state_lock (Lock/Mutex):** Garante exclusão mútua na impressão
- **with state_lock:** Sintaxe que adquire e libera automaticamente
- Resultado: Saída limpa e legível

8.4 - Sincronização de Dados

O dicionário **programmer_states** é acessado por múltiplas threads, portanto é protegido pelo lock.

- **Máximo 1 programador compilando:** Validar exclusão mútua
- **Máximo 2 acessando BD:** Validar semáforo de 2
- **Sem travamentos:** Programa continua rodando
- **Sem saída confusa:** Mensagens aparecem legíveis
- **Fairness:** Todos os programadores conseguem compilar

CONCLUSÃO

Este programa demonstra com sucesso os conceitos fundamentais de **sincronização e exclusão mútua** em sistemas operacionais:

- **Semáforos:** Controlam acesso a múltiplos recursos (compilador e BD)
- **Locks/Mutexes:** Garantem seções críticas seguras
- **Deadlock Prevention:** Ordem fixa de aquisição de recursos
- **Starvation Prevention:** Fila FIFO do Python previne inanição
- **Threads:** Permitem paralelismo real em operações concorrentes
- **Thread Safety:** Variáveis compartilhadas protegidas por sincronização

A solução garante que todos os 5 programadores possam compilar seus módulos de forma ordenada e justa, respeitando as restrições de recursos compartilhados.